Kubernetes Ecosystem Question Index - All 55 Questions

Q#	Question
Q1	Helm basics - charts, releases, repositories
Q2	Helm chart structure - Chart.yaml, values.yaml, templates/
Q3	Helm commands - install, upgrade, rollback, uninstall
Q4	Helm values - values.yaml, --set, --values, precedence
Q5	Helm templating - Go templates, .Values, .Release, if/range
Q6	Helm repositories - add, update, search, Artifact Hub
Q7	Kustomize basics - base, overlays, patches, no templating
Q8	Kustomize vs Helm - when to use which
Q9	Kustomize overlays - dev/staging/prod customization
Q10	Kustomize patches - strategic merge, JSON patch
Q11	Kustomize generators - ConfigMap, Secret generators
Q12	ArgoCD basics - GitOps, application CRD, sync
Q13	FluxCD basics - GitOps toolkit, controllers
Q14	ArgoCD vs FluxCD - comparison
Q15	OpenShift basics - Routes, Projects, SCC
Q16	Helm hooks - pre/post install/upgrade, jobs
Q17	Helm dependencies - subcharts, conditions
Q18	Helm library charts - reusable templates
Q19	Helm testing - helm test, verification
Q20	Helm secrets - SOPS, helm-secrets, Vault
Q21	Helm best practices - versioning, linting
Q22	Helm in CI/CD - helmfile, automated upgrades

Q23	Kustomize components - cross-cutting features
Q24	Kustomize with Helm - post-rendering
Q25	ArgoCD App of Apps - multi-app management
Q26	ArgoCD ApplicationSet - generators, templates
Q27	ArgoCD sync - auto, manual, waves, windows
Q28	ArgoCD RBAC - projects, roles, SSO
Q29	FluxCD Kustomization - reconciliation, health
Q30	FluxCD HelmRelease - automated Helm deploys
Q31	Istio basics - sidecar, VirtualService, Gateway
Q32	Istio traffic - canary, blue-green, circuit breaker
Q33	Istio security - mTLS, AuthorizationPolicy
Q34	Linkerd - lightweight mesh, observability
Q35	Istio vs Linkerd - comparison
Q36	OpenShift advanced - OLM, ImageStreams, Builds
Q37	Helm at scale - chart museum, OCI registry
Q38	GitOps architecture - repo structure, promotion
Q39	GitOps secrets - Sealed Secrets, ESO, SOPS
Q40	Service mesh at scale - overhead, ambient mesh
Q41	ArgoCD multi-cluster - managing 50+ clusters
Q42	Progressive delivery - Argo Rollouts, Flagger
Q43	OpenShift vs vanilla Kubernetes
Q44	K8s ecosystem tool selection framework
Q45	Helm stuck in PENDING_UPGRADE
Q46	Helm rollback lost PVC data

Q47	Helm values override not working
Q48	Kustomize patch not applying
Q49	ArgoCD stuck OutOfSync
Q50	ArgoCD self-heal reverted production hotfix
Q51	Istio sidecar broke application
Q52	Service mesh latency overhead +50ms
Q53	OpenShift SCC blocking Pod
Q54	GitOps repo structure debate
Q55	Argo Rollouts canary failed - auto rollback

SECTION 1 - Junior Core Tools (Q1-Q15)

Q1 Helm basics - charts, releases, repositories

The Simple Answer

Helm is the package manager for Kubernetes. Like apt installs packages on Ubuntu, Helm installs applications on Kubernetes. A Helm chart is a package containing all the Kubernetes manifests (Deployments, Services, ConfigMaps) needed to run an application. A release is a specific installation of a chart in a cluster.

The Analogy - Restaurant Menu

> A Helm chart is a recipe - it defines what ingredients (Kubernetes resources) are needed and how to prepare them. Values are customization options - "spicy level: medium, extra cheese: true." A release is the actual dish served - one recipe can produce multiple dishes with different customizations. A repository is the cookbook - a collection of recipes available to install.

Key Concepts

Chart - A package. Contains templates (YAML with placeholders), a default values.yaml, metadata (Chart.yaml), and optionally helper functions and subcharts. **Release** - An instance of a chart installed in a cluster. helm install my-app ./my-chart creates a release named "my-app." You can install the same chart multiple times with different release names. **Repository** - A server hosting charts. helm repo add bitnami https://charts.bitnami.com/bitnami. Public: Artifact Hub (the central chart discovery site). Private: ChartMuseum, Harbor, OCI registries.

Q2 Helm chart structure

Directory Layout

Chart.yaml - Chart metadata: name, version (chart version), appVersion (application version), dependencies, description. **values.yaml** - Default configuration values. Users override these during installation. **templates/** - Kubernetes manifest templates with Go template syntax. deployment.yaml, service.yaml, ingress.yaml, configmap.yaml, _helpers.tpl (reusable template functions). **charts/** - Dependency charts (subcharts). Downloaded by helm dependency update. **templates/NOTES.txt** - Post-installation message shown to users. **.helmignore** - Files to exclude from the chart package.

Template Rendering

Templates use Go template syntax: {{ .Values.replicaCount }}, {{ .Release.Name }}, {{ .Chart.AppVersion }}. Helm renders templates by substituting values, then applies the rendered YAML to Kubernetes. helm template my-app ./my-chart renders locally without installing - useful for debugging.

Q3 Helm commands

Essential Commands

helm install my-release ./chart - Install a chart as a new release. **helm upgrade my-release ./chart** - Update an existing release with new values or chart version. **helm rollback my-release 1** - Rollback to revision 1. **helm uninstall my-release** - Remove all resources created by the release. **helm list** - List all releases in the current namespace. **helm status my-release** - Show release status, notes, and resource summary. **helm history my-release** - Show all revisions with dates and status. **helm get values my-release** - Show the values used for the current revision. **helm get manifest my-release** - Show the rendered Kubernetes manifests. **helm template my-release ./chart** - Render templates locally without installing. **helm lint ./chart** - Check chart for errors and best practice violations.

The upgrade workflow: Edit values.yaml → helm diff upgrade my-release ./chart (preview changes with helm-diff plugin) → helm upgrade my-release ./chart --atomic (auto-rollback on failure). The --atomic flag is essential for production - if any resource fails to deploy, the entire upgrade is rolled back automatically.

Q4 Helm values - precedence

The Simple Answer

Values customize a chart's behavior. Multiple sources of values are merged with a specific precedence order (highest wins): --set flags → --values file (-f) → parent chart values → subchart values.yaml defaults.

Providing Values

Default: values.yaml in the chart (lowest precedence - designed to be overridden). **File override:** helm install -f production-values.yaml (overrides defaults). **Multiple files:** helm install -f base.yaml -f production.yaml (later files override earlier). **Command-line:** helm install --set image.tag=v2.1.0 (highest precedence - overrides everything). **Structured set:** helm install --set ingress.hosts[0].host=api.example.com.

Best practice: Keep values.yaml as sensible defaults. Create environment-specific files: values-dev.yaml, values-staging.yaml, values-production.yaml. Use --set only for dynamic values from CI/CD (image tags, build numbers). Never put secrets in values files - use helm-secrets or external secret management.

Q5 Helm templating - Go templates

The Simple Answer

Helm templates use Go's template language to generate Kubernetes YAML dynamically. Placeholders are replaced with values during rendering.

Essential Syntax

Value access: {{ .Values.replicaCount }} reads from values.yaml. **Built-in objects:** {{ .Release.Name }} (release name), {{ .Release.Namespace }}, {{ .Chart.AppVersion }}, {{ .Capabilities.KubeVersion }}.

Conditionals: {{- if .Values.ingress.enabled }} ... {{- end }}. Only include Ingress resources when ingress is enabled.

Loops: {{- range .Values.env }} - name: {{ .name }} value: {{ .value }} {{- end }}. Generate multiple environment variables from a list.

Default values: {{ .Values.timeout | default "30s" }}. Use 30s if not specified.

Include helpers: {{ include "mychart.fullname" . }}. Call a named template from _helpers.tpl for reusable logic (generating consistent names, labels).

Whitespace control: {{- (dash before) trims whitespace before the tag. -}} trims after. Essential for clean YAML output - without whitespace control, templates produce YAML with extra blank lines that Kubernetes rejects.

Q6 Helm repositories

The Simple Answer

Helm repositories host charts for distribution. helm repo add name URL registers a repository. helm search repo nginx searches all added repositories for nginx charts.

Key Repositories

Bitnami - The largest collection of production-ready charts. PostgreSQL, Redis, Nginx, Kafka, Elasticsearch. Well-maintained, regularly updated. **Artifact Hub** - Centralized discovery site for Helm charts from all publishers. Search at artifacthub.io. **OCI Registries** - Modern approach. Store charts in container registries (ECR, GCR, Harbor, Docker Hub) alongside images. helm push and helm pull with OCI URLs: oci://registry.example.com/charts/myapp.

Repository Management

helm repo update - Refresh all repo indexes (like apt update). helm search repo nginx --versions - Show all available versions. helm pull bitnami/nginx --version 15.0.0 - Download chart for local inspection.

Q7 Kustomize basics - base, overlays, no templating

The Simple Answer

Kustomize customizes Kubernetes manifests WITHOUT templates. You write plain YAML manifests (base) and layer modifications on top (overlays). No {{ }} syntax, no rendering step - just standard YAML with strategic merge patches.

The Analogy

> Helm is like a recipe with blanks you fill in: "Add {{ .Values.sugar }} cups of sugar." Kustomize is like ordering a base meal and adding customizations: "Start with the standard burger (base), add extra cheese and no pickles (overlay)." The base burger is a real, complete burger - not a template with holes.

Structure

base/ - Original, unmodified manifests: deployment.yaml, service.yaml, kustomization.yaml (lists the resources). **overlays/dev/** - Dev customization: kustomization.yaml (references base, adds patches), patches/replicas.yaml. **overlays/prod/** - Prod customization: different patches, different namespace, different resource limits.

Building

kubectl apply -k overlays/prod/ applies the base manifests with production patches. Or: kustomize build overlays/prod/ | kubectl apply -f - renders and applies.

> **Kustomize is built into kubectl.** No separate tool installation needed (since kubectl 1.14). This is Kustomize's biggest adoption advantage over Helm - zero additional tooling.

Q8 Kustomize vs Helm - when to use which

The Simple Answer

Feature	Helm	Kustomize
Approach	Templating (generate YAML)	Patching (modify YAML)
Learning curve	Medium (Go templates)	Low (plain YAML)
Packaging	Charts with versioning	Directories with bases/overlays
Installation	Separate tool (helm)	Built into kubectl

Release management	Built-in (revisions, rollback)	None (just YAML)
Community charts	Thousands available	No package ecosystem
Complexity handling	Powerful for complex apps	Better for simple customization
Best for	Third-party apps, complex deployments	Internal apps, env-specific patches

When to Use Helm

Installing third-party software (databases, monitoring stacks, Ingress controllers). Complex applications with many configuration options. When you need release management (rollback, history). When the community chart exists and is well-maintained.

When to Use Kustomize

Simple internal applications with minor env-specific differences. Teams that want plain YAML without templating complexity. When kubectl is the only tool available. Small projects where Helm's overhead is not justified.

Using Both Together

Many teams use Helm for third-party charts and Kustomize for internal applications. ArgoCD and FluxCD support both natively. You can even use Kustomize to post-render Helm output - see Q24.

Q9 Kustomize overlays - environment customization

The Simple Answer

Overlays modify the base manifests for specific environments without changing the originals. Each overlay has a kustomization.yaml that references the base and adds modifications.

Example Structure

base/deployment.yaml - 1 replica, no resource limits, latest image tag. overlays/dev/ - 1 replica, low resource limits, dev namespace. overlays/staging/ - 2 replicas, medium limits, staging namespace. overlays/prod/ - 5 replicas, high limits, prod namespace, HPA enabled.

What Overlays Can Do

Change namespace: namespace: production in kustomization.yaml. Add labels: commonLabels: env: production. Add name prefix/suffix: namePrefix: prod-. Change image tags: images: [{name: myapp, newTag: v2.1.0}]. Add or modify resources: patches and additional resource files.

Q10 Kustomize patches - strategic merge, JSON patch

Strategic Merge Patch

Merges your patch with the original resource by matching the resource kind, name, and field paths. You only specify the fields you want to change - everything else remains untouched.

Example patch: apiVersion: apps/v1, kind: Deployment, metadata: name: my-app, spec: replicas: 5. This changes ONLY the replica count. All other deployment fields remain from the base.

JSON Patch (RFC 6902)

Precise operations: add, remove, replace, move, copy, test. More explicit than strategic merge. Used when strategic merge does not produce the desired result (e.g., modifying a specific array element by index). Example: op: replace, path: /spec/replicas, value: 5.

Use strategic merge for most cases - it is simpler and more readable. Use JSON patch when you need to modify specific array elements or when strategic merge's merge behavior is not what you want.

Q11 Kustomize generators

ConfigMapGenerator

Generates ConfigMaps from files, literals, or env files. Automatically appends a content hash to the name (my-config-abc123). When the content changes, the name changes, triggering a rolling update of Pods referencing it. This solves the "changed ConfigMap but Pods did not restart" problem.

SecretGenerator

Same as ConfigMapGenerator but for Secrets. secretGenerator: [{name: db-creds, literals: [username=admin, password=secret123]}]. The secret name gets a hash suffix, ensuring Pods restart when secrets change.

Q12 ArgoCD basics - GitOps operator

The Simple Answer

ArgoCD is a GitOps continuous delivery tool for Kubernetes. It watches a Git repository and ensures the cluster state matches the desired state defined in Git. When you push a change to Git, ArgoCD detects the difference and syncs the cluster.

Key Concepts

Application CRD - Defines: source (Git repo, path, branch/tag), destination (cluster, namespace), and sync policy. **Sync** - Applying the Git state to the cluster. Manual sync requires a click/command. Auto-sync happens automatically on Git changes. **Self-Heal** - When someone modifies a resource directly (kubectl edit), ArgoCD detects the drift and reverts to the Git state. **Prune** - When a resource is removed from Git, ArgoCD deletes it from the cluster.

Architecture

API Server - Manages applications, handles UI/CLI requests. **Repo Server** - Clones Git repos, renders Helm/Kustomize manifests. **Application Controller** - Watches applications, detects drift, triggers sync. **Redis** - Caching. **Dex** - SSO authentication.

Why ArgoCD dominates GitOps: Rich UI (visual application topology, real-time sync status, log streaming), multi-cluster support, SSO/RBAC, ApplicationSets for templating, and the largest GitOps community.

Q13 FluxCD basics - GitOps toolkit

The Simple Answer

FluxCD is a set of Kubernetes controllers that implement GitOps. Unlike ArgoCD (monolithic application), Flux is modular - each controller handles a specific responsibility.

Controllers

Source Controller - Watches Git repositories, Helm repositories, S3 buckets for changes. **Kustomize Controller** - Applies Kustomize manifests from the source. Reconciles periodically. **Helm Controller** - Manages HelmRelease CRDs. Installs, upgrades, and rolls back Helm charts. **Notification Controller** - Sends alerts (Slack, Teams,

webhooks) on events. **Image Automation Controller** - Detects new container image tags and updates Git manifests automatically.

Key Difference from ArgoCD

Flux is CLI-first (no built-in UI - use Weave GitOps UI or Grafana dashboards). Flux uses Kubernetes CRDs natively - everything is a Kubernetes resource. Flux's image automation (auto-updating image tags in Git) is built-in; ArgoCD needs Argo CD Image Updater (separate project).

Q14 ArgoCD vs FluxCD

The Simple Answer

Feature	ArgoCD	FluxCD
Architecture	Monolithic application	Modular controllers
UI	Rich built-in UI	No built-in UI (Weave GitOps)
Multi-cluster	Native (manage from one ArgoCD)	Federation (Flux per cluster)
Helm support	Via Application CRD	Native HelmRelease CRD
Kustomize	Native	Native
Image automation	Separate tool	Built-in
RBAC	AppProject-based	Kubernetes RBAC-native
Learning curve	Lower (UI helps)	Higher (CLI/CRD-only)
Community	Larger	Smaller but growing
Best for	Multi-cluster, teams wanting UI	Single cluster, K8s-native teams

The honest recommendation: ArgoCD for most teams - the UI dramatically reduces the learning curve and provides instant visibility into sync status across clusters. FluxCD for teams that are deeply Kubernetes-native and prefer everything as CRDs without a separate UI.

Q15 OpenShift basics - Routes, Projects, SCC

The Simple Answer

OpenShift is Red Hat's enterprise Kubernetes platform. It adds features on top of vanilla Kubernetes: built-in CI/CD, developer portal, integrated registry, enhanced security (SCCs), and enterprise support.

Key Additions Over Kubernetes

Routes - OpenShift's Ingress alternative (predates Kubernetes Ingress). Exposes services externally with automatic TLS. oc expose service my-app creates a Route. **Projects** - Enhanced namespaces with RBAC, quotas, and network isolation. oc new-project my-app creates a project. **Security Context Constraints (SCCs)** - Control

what a Pod is allowed to do: run as root, access host network, use volumes. More restrictive than Kubernetes Pod Security Admission. Default SCC (restricted) blocks running as root - many Docker Hub images fail without adjustment. See scenario Q53. **BuildConfig** - Build container images directly on the cluster. Source-to-Image (S2I) builds from source code without a Dockerfile. **ImageStreams** - Track image tags and trigger deployments on new image pushes. **Operator Lifecycle Manager (OLM)** - Install, update, and manage Operators from a catalog.

SECTION 2 - Mid-Level Production Patterns (Q16-Q36)

Q16 Helm hooks - pre/post install/upgrade

The Simple Answer

Hooks run jobs at specific points in the release lifecycle. They execute BEFORE or AFTER install, upgrade, rollback, delete, and test operations.

Common Use Cases

pre-install/pre-upgrade - Run database migrations before deploying new code. Create prerequisite resources. Validate configuration. **post-install/post-upgrade** - Send deployment notifications. Register service with service discovery. Run smoke tests. **pre-delete** - Backup data before uninstalling. Deregister from load balancer.

How Hooks Work

Add an annotation to any Kubernetes resource: helm.sh/hook: pre-upgrade. Add weight for ordering: helm.sh/hook-weight: "1" (lower runs first). Add delete policy: helm.sh/hook-delete-policy: before-hook-creation (clean up previous hook resources). Hook resources are NOT managed by the release - they are created, executed, and optionally deleted independently.

Q17 Helm dependencies - subcharts

The Simple Answer

Charts can depend on other charts. An application chart can include PostgreSQL, Redis, and Nginx as dependencies. Dependencies are listed in Chart.yaml: dependencies: [{name: postgresql, version: "12.x.x", repository: "https://charts.bitnami.com/bitnami"}].

Managing Dependencies

helm dependency update - Downloads dependencies into charts/ directory and generates Chart.lock (pinned versions). **Conditional dependencies** - condition: postgresql.enabled allows users to disable the dependency: postgresql.enabled: false in values.

Subchart Values

Parent chart can override subchart values: postgresql: auth: postgresPassword: mypassword in the parent's values.yaml. The subchart name becomes the key prefix.

Q18 Helm library charts

The Simple Answer

Library charts contain reusable template definitions but do NOT deploy any resources themselves. They define helper functions (labels, selectors, container specs) that other charts import and use. This eliminates copy-pasting the same template logic across 20 microservice charts.

Pattern

Create a library chart with type: library in Chart.yaml. Define templates in _helpers.tpl: common labels, standard deployment template, standard service template. Application charts declare the library as a dependency and call its templates: {{ include "common.deployment" . }}.

Q19 Helm testing

The Simple Answer

helm test my-release runs test hooks - Pods annotated with helm.sh/hook: test. Tests verify the release is working: a Pod that runs curl against the service and exits 0 on success, or runs a database connectivity check.

In CI/CD

Deploy to staging → helm test → if tests pass, promote to production. Tests catch: incorrect configuration, failed database migrations, broken service connectivity. Tests are defined in templates/tests/ within the chart.

Q20 Helm secrets - SOPS, helm-secrets

The Simple Answer

Secrets in values.yaml committed to Git in plaintext is a security violation. Helm-secrets plugin encrypts values files using SOPS (Secrets OPerationS) with AWS KMS, GCP KMS, Azure Key Vault, or PGP keys.

Workflow

helm secrets encrypt values-secrets.yaml encrypts the file in place. The encrypted file is committed to Git. helm secrets install my-release ./chart -f values-secrets.yaml decrypts at install time. Developers with the right KMS/PGP access can encrypt and decrypt. CI/CD pipelines use IAM roles (AWS) or service accounts (GCP) for decryption.

Alternative: External Secrets

Instead of encrypting values files, reference secrets from Vault or cloud secret managers at runtime. External Secrets Operator syncs external secrets into Kubernetes Secrets. The Helm chart references the K8s Secret - no secrets in values files at all.

Q21 Helm best practices

The Simple Answer

Version charts independently - Chart version (version: in Chart.yaml) and app version (appVersion:) should both follow semver. Bump chart version on ANY template change. **Use --atomic for production** - Auto-rollback on failed upgrade. **Always lint** - helm lint ./chart catches template errors, missing values, and best practice violations. **Pin dependency versions** - Use Chart.lock. Never use "*" for dependency versions. **Use _helpers.tpl** - Define reusable templates for labels, names, and selectors. Do not repeat the same label block in every template. **Document values** - Comment every value in values.yaml with description and default. **helm diff before upgrade** - Preview changes with the helm-diff plugin before applying.

Q22 Helm in CI/CD - helmfile

Helmfile

Declaratively manage multiple Helm releases across environments. A helmfile.yaml defines: releases (chart + values per environment), repositories, and hooks. helmfile sync installs/upgrades all releases. helmfile diff shows what would change. Helmfile supports: environment-specific values ({{ .Environment.Name }}), secret decryption (SOPS), and dependency ordering.

ArgoCD App-of-Apps

Define a parent ArgoCD Application that points to a directory of child Application manifests. Add a new microservice → add an Application YAML to the directory → ArgoCD automatically discovers and deploys it. Self-managing: the parent app manages child apps, child apps manage workloads.

Q23 Kustomize components

The Simple Answer

Components are reusable Kustomize packages that add cross-cutting features to any base. Unlike overlays (which customize a specific base), components can be applied to multiple different bases. Example: a "monitoring" component adds Prometheus annotations and a sidecar to any deployment it is applied to.

Q24 Kustomize with Helm - post-rendering

The Simple Answer

helm template renders a Helm chart to plain YAML. Pipe the output through Kustomize for additional modifications: helm template my-release ./chart | kustomize build overlay/. Or use Helm's built-in post-renderer: helm install my-release ./chart --post-renderer kustomize.

Why Combine Them

The Helm chart sets up 90% of the configuration. Kustomize handles the last 10%: adding organization-specific labels, injecting sidecars, modifying resource limits for specific environments, or adding annotations that the chart does not support.

Q25 ArgoCD App of Apps

The Simple Answer

A parent Application points to a Git directory containing child Application manifests. Each child Application deploys a different microservice. Adding a service = adding a YAML file. Removing a service = deleting the file.

The Pattern

apps/ directory contains: app-payment.yaml (ArgoCD Application for payment service), app-orders.yaml, app-notifications.yaml. Parent Application: source path = apps/. ArgoCD creates all child applications. Each child application manages its own Helm chart or Kustomize overlay.

Benefits

One Git PR adds a new service to the platform. The platform is self-describing - the apps/ directory IS the list of services. ArgoCD manages the entire platform declaratively from a single entry point.

Q26 ArgoCD ApplicationSet - generators

The Simple Answer

ApplicationSet generates multiple ArgoCD Applications from a template. Instead of writing 50 Application manifests for 50 microservices, write one ApplicationSet with a generator that produces all 50.

Generators

List generator - Explicit list of applications with parameters. **Git generator** - Scan a Git directory for subdirectories, each becomes an application. **Cluster generator** - One application per registered cluster. **Matrix generator** - Combine two generators (every service × every cluster). **Pull Request generator** - Create a preview environment for every open PR.

Example

Git generator scans microservices/ directory. Each subdirectory (payment/, orders/, notifications/) becomes an ArgoCD Application. Adding a new microservice = creating a new subdirectory in Git. ArgoCD auto-discovers and deploys it.

Q27 ArgoCD sync strategies

The Simple Answer

Manual sync - ArgoCD detects drift but waits for a human to click "Sync." Used for: production with change management approval. **Auto-sync** - ArgoCD automatically applies changes when Git changes. Used for: dev, staging, and production with strong CI/CD gates.

Sync Waves

Control the order of resource creation. wave 0: namespace, secrets, configmaps. wave 1: databases, caches. wave 2: application deployments. wave 3: ingress, monitoring. Annotate resources: argocd.argoproj.io/sync-wave: "1".

Sync Windows

Restrict when syncs can occur. "Allow auto-sync only during business hours." "Block syncs during holiday freeze." Defined per ArgoCD project.

Self-Heal

Reverts manual cluster changes to match Git. If someone runs kubectl edit to change replicas, ArgoCD detects the drift and reverts to the Git-defined value. Essential for drift prevention. See scenario Q50 for when this causes problems.

Q28 ArgoCD RBAC - projects, roles

Projects

AppProjects scope what applications can access. Define: allowed source repos, allowed destination clusters and namespaces, allowed resource kinds. The "payments" project can only deploy from the payments repo to the payments namespace. This prevents: cross-team interference, accidental production deployments, unauthorized cluster access.

RBAC

ArgoCD has its own RBAC system (separate from Kubernetes RBAC). Policies define: who (role) can do what (action) on which (resource). Example: p, role:payments-deployer, applications, sync, payments/*, allow. Integrate with SSO (OIDC, SAML, LDAP) - map SSO groups to ArgoCD roles.

Q29 FluxCD Kustomization - reconciliation

The Simple Answer

A Flux Kustomization CRD defines: which source to watch (GitRepository, Bucket), which path to apply, and how often to reconcile (interval: 5m). Every 5 minutes, Flux checks if the cluster state matches the Git state. If it drifts, Flux corrects it.

Health Checks

Flux can wait for deployed resources to become healthy before considering the sync successful. If a Deployment's Pods fail to start, Flux reports the Kustomization as unhealthy. Configure: timeout for health assessment, dependency ordering between Kustomizations.

Q30 FluxCD HelmRelease

The Simple Answer

HelmRelease CRD declaratively manages Helm chart installations. Define: chart source (HelmRepository or GitRepository), values, upgrade and rollback policies. Flux automatically installs, upgrades, and rolls back Helm releases based on the CRD.

Features

Automatic upgrades - When a new chart version appears in the repository, Flux upgrades. semver ranges control which versions are accepted: ">=1.0.0 <2.0.0". **Automatic rollback** - If an upgrade fails health checks, Flux rolls back to the previous version. **Values from ConfigMaps/Secrets** - Reference Kubernetes ConfigMaps or Secrets as value sources. Enables dynamic values without storing them in Git.

Q31 Istio basics - sidecar, VirtualService, Gateway

The Simple Answer

Istio is a service mesh that adds traffic management, security, and observability to microservices by injecting a sidecar proxy (Envoy) into every Pod.

Key Resources

VirtualService - Defines routing rules. Route 90% traffic to v1, 10% to v2 (canary). Route based on HTTP headers (A/B testing). Set timeouts, retries, fault injection. **DestinationRule** - Defines policies for traffic TO a service. Connection pooling, circuit breaker, TLS settings, load balancing algorithm. **Gateway** - Manages inbound traffic to the mesh. Like Ingress but more powerful. Configures ports, TLS, and hostname routing. **PeerAuthentication** - Configures mTLS mode: STRICT (all traffic must be mTLS), PERMISSIVE (accept both), DISABLE. **AuthorizationPolicy** - Access control: which services can call which other services. "Only the frontend can call the API. Only the API can call the database."

Sidecar Injection

Add the label istio-injection: enabled to a namespace. Every new Pod in that namespace automatically gets an Envoy sidecar container. The sidecar intercepts all traffic (inbound and outbound) transparently.

Q32 Istio traffic management - canary, circuit breaker

Canary Deployment

VirtualService routes 95% of traffic to v1 and 5% to v2. Monitor v2's error rate and latency. If healthy, gradually increase: 10%, 25%, 50%, 100%. If unhealthy, route 100% back to v1. No infrastructure changes - just VirtualService configuration.

Circuit Breaker

DestinationRule defines: maxConnections: 100, maxPendingRequests: 50, consecutiveErrors: 5. After 5 consecutive errors, the circuit opens - traffic is rejected immediately without calling the failing service. After a cooldown period, the circuit half-opens - a test request is sent. If successful, the circuit closes. This prevents cascade failures.

Fault Injection

Inject delays (simulate slow responses) or aborts (simulate errors) to test resilience. VirtualService: fault: delay: percentage: 10, fixedDelay: 5s. 10% of requests to the service are delayed by 5 seconds. Used for: chaos engineering, testing circuit breakers, verifying timeout configuration.

Q33 Istio security - mTLS, AuthorizationPolicy

mTLS in Istio

Istio's mTLS is automatic. Istiod (the control plane) acts as a certificate authority. It issues certificates to every sidecar. Sidecars automatically encrypt all inter-service traffic with mTLS. Certificate rotation is automatic (24-hour default). STRICT mode enforces mTLS - non-mesh traffic is rejected. PERMISSIVE mode accepts both mTLS and plaintext (useful during migration).

AuthorizationPolicy

Fine-grained access control. Example: allow only the frontend service to call the api service on GET /v1/products. Deny all traffic to the database except from the api service. This is network segmentation at the application layer - more precise than NetworkPolicies (which operate at L3/L4).

Q34 Linkerd - lightweight mesh

The Simple Answer

Linkerd is a lightweight, simple service mesh. Uses its own micro-proxy (linkerd2-proxy, written in Rust) instead of Envoy. Smaller footprint (10 MB per proxy vs 50+ MB for Envoy). Simpler configuration - fewer knobs, less complexity. Automatic mTLS, traffic splitting, retries, timeouts, and observability out of the box.

Linkerd vs Istio in One Line

> Linkerd is "it just works" with less configuration. Istio is "you can configure everything" with more complexity. Linkerd for simple mesh needs. Istio for advanced traffic management.

Q35 Istio vs Linkerd

The Simple Answer

Feature	Istio	Linkerd
Proxy	Envoy (C++)	linkerd2-proxy (Rust)
Memory per sidecar	~50-100 MB	~10-20 MB
Latency overhead	~2-5ms p99	~1-2ms p99
Configuration	Complex (many CRDs)	Simple (minimal config)
Traffic management	Advanced (canary, fault injection, mirrors)	Basic (traffic split, retries)
mTLS	Automatic	Automatic
Multi-cluster	Native	Supported (multi-cluster)
Learning curve	Steep	Gentle
Best for	Complex routing, enterprise	Simple mesh, smaller teams

Q36 OpenShift advanced - OLM, ImageStreams, Builds

Operator Lifecycle Manager (OLM)

Manages Kubernetes Operators. A catalog of pre-built Operators (Red Hat, community). Install Operators from the catalog with one click. OLM handles: installation, upgrades, RBAC for the Operator, CRD management.

ImageStreams

An abstraction over container image references. An ImageStream tracks image tags and triggers actions (new builds, new deployments) when a tag changes. Push a new image → ImageStream detects it → triggers a deployment. Like a webhook but built into OpenShift.

Build Strategies

Source-to-Image (S2I) - Push source code, OpenShift builds the container image automatically. No Dockerfile needed. S2I builder images detect the language and build accordingly. **Docker build** - Standard Dockerfile-based build on the cluster. **Custom build** - Your own build logic in a container. Builds run ON the cluster - no external CI needed for simple cases. For complex CI, integrate with Jenkins, Tekton, or GitHub Actions.

SECTION 3 - Senior Architecture & Scale (Q37-Q44)

Q37 Helm at scale - chart museum, OCI

The Simple Answer

At scale (50+ microservices, each with its own chart), you need: a private chart repository (ChartMuseum or OCI registry), versioned charts with CI/CD (lint, test, package, publish on every change), and a library chart for shared templates.

OCI Registries for Charts

Modern approach: store charts in the same registry as images. helm push mychart-1.0.0.tgz oci://registry.example.com/charts/. Benefits: unified authentication, single registry to manage, leverages existing registry infrastructure (ECR, Harbor, GCR).

Q38 GitOps architecture - repo structure, promotion

Monorepo vs Polyrepo

App repo + config repo (recommended): application source code in one repo, Kubernetes manifests in another. CI builds the image and updates the tag in the config repo. ArgoCD/Flux watches the config repo. Clean separation of concerns.

Monorepo - Everything (source + manifests) in one repo. Simpler for small teams. ArgoCD watches a specific path within the repo.

Environment Promotion

Directory-based: config-repo/envs/dev/, config-repo/envs/staging/, config-repo/envs/prod/. Promote by copying/merging from dev to staging to prod. **Branch-based:** dev branch → staging branch → main branch. ArgoCD watches different branches per environment. Promote by merging branches.

> **The recommendation:** Directory-based promotion with a single branch (main). Each environment is a directory with Kustomize overlays. Promotion is a PR that updates the image tag in the next environment's overlay. PRs provide review, audit trail, and approval gates.

Q39 GitOps secrets

The Options

Sealed Secrets - Encrypt secrets with a cluster-specific key. Commit encrypted SealedSecret to Git. The controller decrypts in-cluster. Simple. Cluster-specific (cannot decrypt on another cluster without the key).

External Secrets Operator (ESO) - Syncs secrets from external stores (Vault, AWS Secrets Manager, GCP Secret Manager) into Kubernetes Secrets. The external store is the source of truth. No secrets in Git - only references.

SOPS with ArgoCD - Encrypt values files with SOPS (using KMS). ArgoCD decrypts at sync time using the kustomize-sops or helm-secrets plugin. Secrets are encrypted in Git - readable only with KMS access.

> **The recommendation:** ESO for teams with Vault or cloud secret managers. Sealed Secrets for simpler setups without external secret stores. SOPS for Helm-heavy workflows where values files need encryption.

Q40 Service mesh at scale - overhead, ambient mesh

Performance Overhead

Every sidecar adds: 50-100 MB memory (Envoy), 1-5ms latency per hop. In a 100-service architecture with 5 hops per request: 5-25ms added latency. For most web applications, this is acceptable. For low-latency systems (trading, gaming), it may not be.

Istio Ambient Mesh

New architecture (from Istio 1.15+) that eliminates per-Pod sidecars. Instead: a shared L4 proxy (ztunnel) runs per node, handling mTLS and L4 policies. An optional L7 proxy (waypoint) is deployed per service for advanced traffic management. Benefits: no sidecar injection, lower resource usage, simpler operations. The future direction of Istio.

Q41 ArgoCD multi-cluster

The Simple Answer

A single ArgoCD installation can manage applications across 50+ clusters. Each cluster is registered with ArgoCD: argocd cluster add context-name. Applications specify the destination cluster. The ArgoCD UI shows all clusters and applications in one view.

Architecture for Scale

Hub-spoke model - One management cluster runs ArgoCD. Spoke clusters are registered as destinations. ArgoCD reaches spoke clusters via the Kubernetes API (requires network connectivity and credentials). **ApplicationSets** - Generate one application per cluster using the cluster generator. A new cluster is registered → ApplicationSet automatically creates applications for it.

Q42 Progressive delivery - Argo Rollouts, Flagger

Argo Rollouts

Replaces Kubernetes Deployment with a Rollout CRD that supports: **Canary** - Gradually shift traffic (5% → 10% → 25% → 50% → 100%) with analysis at each step. Integration with Istio, Nginx, ALB for traffic splitting. Automated analysis: query Prometheus for error rate - if > 1%, abort and rollback. **Blue-Green** - Deploy new version alongside old. Switch traffic atomically. Instant rollback by switching back.

Flagger

Automates canary deployments with any service mesh (Istio, Linkerd, Nginx, Contour). Progressive traffic shifting with automated analysis. Webhooks for custom checks. Works with Flux (native integration).

Q43 OpenShift vs vanilla Kubernetes

When OpenShift Is Worth It

Your organization requires: enterprise support (24/7 Red Hat support), built-in security (SCCs are more restrictive than default K8s), integrated developer experience (web console, S2I builds), compliance certifications (FedRAMP, FIPS), Operator ecosystem (OperatorHub), or multi-cluster management (RHACM).

When Vanilla Kubernetes Is Better

Cost-sensitive (OpenShift licensing is expensive). Already invested in custom tooling. Running on managed K8s (EKS, GKE, AKS already provide management). Small team that does not need enterprise features. Prefer flexibility over opinionated platform.

Q44 K8s ecosystem tool selection

The Decision Framework

Package management: Use Helm for third-party charts. Kustomize for internal apps. Both for maximum flexibility. **GitOps:** ArgoCD for multi-cluster, team with UI preference. Flux for single-cluster, Kubernetes-native teams. **Service mesh:** Start without one. Add Linkerd when you need mTLS and basic observability. Consider Istio when you need advanced traffic management. **Progressive delivery:** Argo Rollouts with ArgoCD. Flagger with Flux. **Secrets:** External Secrets Operator + Vault (enterprise). Sealed Secrets (simple setups).

> **The pragmatic rule:** Start simple. Add complexity only when you have a specific problem to solve. A team with 5 services does not need Istio. A team with 50 services across 10 clusters does.

SECTION 4 - Tricky Scenarios (Q45-Q55)

Q45 Helm stuck in PENDING_UPGRADE

What Happened

A helm upgrade was interrupted (CI pipeline timeout, network disconnect, manual Ctrl+C). The release is stuck in PENDING_UPGRADE status. Further upgrades fail because Helm detects the pending operation.

Step-by-Step

Step 1 - Check release status. helm history my-release. The latest revision shows PENDING_UPGRADE.

Step 2 - Rollback to the last successful revision. helm rollback my-release REVISION_NUMBER. This sets the release back to a DEPLOYED state.

Step 3 - If rollback also fails, force it: helm rollback my-release REVISION_NUMBER --force. This deletes and recreates resources rather than patching them.

Step 4 - Nuclear option. If rollback is impossible: helm uninstall my-release (resources remain in the cluster). Then helm install my-release ./chart with the correct values. Check cluster resources before reinstalling - you may need to delete orphaned resources manually.

Prevention

> Always use --timeout (sufficient for your deployment) and --atomic (auto-rollback on failure). Use --wait to ensure Helm waits for all resources to become ready before marking success. In CI/CD, set pipeline timeouts longer than Helm timeouts to avoid interrupting Helm mid-operation.

Q46 Helm rollback lost PVC data

What Happened

A Helm upgrade changed the PVC template (new storage class, new size). After issues, helm rollback was run. The Deployment rolled back to the old Pod spec - but the PVC was NOT rolled back. The old Pod spec references the old PVC name, which no longer exists (or has different properties). The Pod is stuck in Pending.

Why It Happens

Helm rollback reverts Deployment, Service, ConfigMap templates - but PVCs are special. Kubernetes does not allow modifying or deleting PVCs that are in use. Helm cannot rollback a PVC to its previous state.

Fix

Manually recreate the PVC with the old name and properties. Or modify the Deployment to reference the new PVC. Or restore data from backup into a new PVC and update the Deployment.

Prevention

> Never change PVC properties (storage class, access mode) in Helm upgrades without a migration plan. PVCs are not safely rollbackable. Treat PVC changes as data migrations - plan, test, and backup before executing. Separate PVC management from application Helm charts.

Q47 Helm values override not working

Step-by-Step

Step 1 - Check the effective values. helm get values my-release shows what values are actually applied. Compare with what you expected.

Step 2 - Check value names. YAML indentation matters. image.tag in --set must match the YAML structure: image: tag: in values.yaml. A common mistake: --set imageTag=v2 when the chart expects image.tag.

Step 3 - Check precedence. --set overrides -f values file, which overrides defaults. If using multiple -f files, later files override earlier ones.

Step 4 - Debug with template rendering. helm template my-release ./chart -f my-values.yaml --debug shows the rendered output and where values come from.

Q48 Kustomize patch not applying

Common Causes

Resource not matched. Strategic merge patch must match: apiVersion, kind, AND metadata.name. If any differ between the patch and the base resource, the patch creates a NEW resource instead of modifying the existing one.

Wrong patch type. Strategic merge cannot remove array elements (only add or replace). Use JSON patch for removing specific items from arrays.

Kustomization.yaml not listing the patch. The patch file must be referenced in kustomization.yaml under patches: or patchesStrategicMerge:.

Debugging

kustomize build . renders the output. Compare with the expected result. If the patch is not applied, check: resource matching, patch file path, and kustomization.yaml references.

Q49 ArgoCD stuck OutOfSync

Step-by-Step

Step 1 - Check the diff. ArgoCD UI → Application → Diff shows exactly what differs between Git and the cluster.

Step 2 - Common causes. Resource fields that Kubernetes adds automatically (status, metadata.resourceVersion, defaulted fields). ArgoCD sees these as drift. Fix: add ignoreDifferences in the Application spec for fields that are expected to differ.

Step 3 - Webhook or admission controller modifying resources. A mutating webhook adds labels or annotations after ArgoCD applies the manifest. ArgoCD sees the webhook's additions as drift. Fix: ignoreDifferences for the webhook-added fields.

Step 4 - Sync and verify. If the drift is legitimate (someone manually changed the cluster), sync to restore the Git state. If the drift is expected (defaulted fields), configure ignoreDifferences to prevent false OutOfSync.

Q50 ArgoCD self-heal reverted production hotfix

What Happened

An urgent production bug required an immediate fix. The on-call engineer ran kubectl set image deployment/app app=myapp:hotfix. The hotfix worked. 30 seconds later, ArgoCD detected the drift and reverted the image back to the Git-defined version. The bug returned.

Why

ArgoCD self-heal mode continuously reconciles cluster state to Git state. The kubectl change was a drift from Git. ArgoCD "fixed" it by reverting to Git.

Fix

Step 1 - Disable auto-sync temporarily. In ArgoCD UI: Application → Sync Policy → disable auto-sync. Or: argocd app set my-app --sync-policy none.

Step 2 - Apply the hotfix. kubectl set image works now without being reverted.

Step 3 - Commit the fix to Git. Update the image tag in Git to the hotfix version. Push, get reviewed (even as an emergency PR).

Step 4 - Re-enable auto-sync. ArgoCD now sees Git matches the cluster - no drift.

Prevention

> Have a documented emergency process: 1. Disable auto-sync. 2. Apply hotfix. 3. Commit to Git within 1 hour. 4. Re-enable auto-sync. Add a Slack/PagerDuty alert when auto-sync is disabled so it does not stay disabled indefinitely.

Q51 Istio sidecar broke application

Common Causes

Port conflict. The application uses a port that Envoy also uses (15000, 15001, 15006, 15020, 15021, 15090). Fix: change the application port.

Readiness probe failing. The sidecar intercepts traffic. If the application's readiness probe does not account for the sidecar's startup time, the probe fails. Fix: use Istio's holdApplicationUntilProxyStarts: true to delay application start until the sidecar is ready.

Non-HTTP protocols. The sidecar expects HTTP but the application uses a custom TCP protocol. Fix: name the port with the protocol prefix: containerPort: 9000, name: tcp-custom. Istio uses port names to determine protocol handling.

Init container order. Istio's init container (istio-init) sets up iptables rules. If the application's init container needs network access, it may fail because iptio-init has not run yet. Fix: ensure istio-init runs first, or exclude specific ports from sidecar interception.

Q52 Service mesh latency overhead +50ms

Step-by-Step

Step 1 - Confirm the overhead. Compare latency with and without sidecar on the same service. If p99 increases 50ms, that is above normal (expected: 2-5ms per hop).

Step 2 - Check resource limits. The sidecar may be CPU-throttled. Check: kubectl top pod shows sidecar resource usage. Increase sidecar CPU limits if throttled.

Step 3 - Check mTLS overhead. TLS handshake on every request adds latency. Verify connection reuse - the sidecar should reuse TLS connections. Check Envoy stats for new connection rate.

Step 4 - Check access logging. Envoy access logging enabled at DEBUG level adds significant overhead. Set to WARNING or disable for performance-sensitive services.

Step 5 - Consider ambient mesh. Istio ambient mode (ztunnel) has lower overhead than sidecar mode - no per-Pod proxy, shared node-level proxy instead.

Q53 OpenShift SCC blocking Pod

What Happened

A Pod from Docker Hub fails to start on OpenShift with: "unable to validate against any security context constraint." The Pod tries to run as root, but OpenShift's default restricted SCC blocks this.

Step-by-Step

Step 1 - Check the required security context. oc get pod my-pod -o yaml | grep -A20 securityContext. Does it request runAsUser: 0 (root), privileged: true, or volume types not allowed by restricted?

Step 2 - Modify the image. Best fix: change the Dockerfile to use a non-root user (USER 1001). Many Docker Hub images run as root unnecessarily.

Step 3 - If the image cannot be modified, grant a less restrictive SCC: oc adm policy add-scc-to-user anyuid -z my-service-account -n my-namespace. anyuid allows running as any UID including root. Use this sparingly - it weakens security.

Step 4 - For truly privileged workloads (system agents, monitoring), use the privileged SCC. Document why and get security team approval.

Prevention

> Always use non-root containers. Build images with USER instruction. Test on OpenShift before deploying - catch SCC issues in CI, not production. Prefer the restricted SCC (default) - escalate only when absolutely necessary with documented justification.

Q54 GitOps repo structure debate

The Decision

Monorepo (app + config in one repo):

Pros: atomic changes (code + manifest in one PR), simpler tooling, easier onboarding. Cons: ArgoCD must watch specific paths (not entire repo), large repo for many services, CI triggers on every change (need path filtering).

Polyrepo (separate app and config repos):

Pros: clear separation (devs own app repo, platform team owns config repo), CI only triggers on relevant changes, config repo is the GitOps source of truth. Cons: two PRs for one change (update code, then update config), more repos to manage, version coordination.

The Recommendation

> **Separate repos** for most teams. The config repo is the single source of truth for what runs in each environment. CI updates the image tag in the config repo automatically. ArgoCD watches only the config repo. This cleanly separates "what to build" (app repo) from "what to deploy" (config repo). Monorepo works for small teams (<5 developers, <10 services) where the overhead of multiple repos is not justified.

Q55 Argo Rollouts canary failed - automatic rollback

What Happened

An Argo Rollout canary was deployed. 5% of traffic was routed to the new version. The AnalysisRun queried Prometheus for error rate. Error rate exceeded the threshold (>1%). Argo Rollouts automatically aborted the canary and rolled back to the stable version.

Step-by-Step Investigation

Step 1 - Check the AnalysisRun. kubectl get analysisrun shows the analysis result and the metric values that triggered the abort.

Step 2 - Check the canary Pods. kubectl logs for the canary ReplicaSet. Were there application errors, OOM kills, or configuration issues?

Step 3 - Check the metric. Was the error rate real? A small number of errors in 5% traffic can produce a high error rate percentage. Tune the analysis: increase the sample size (run the analysis for longer), adjust the threshold, or use error count instead of error rate for small traffic percentages.

Step 4 - Fix and retry. Fix the issue in the new version. Push a new image. Argo Rollouts starts a new canary automatically.

The Value of Automatic Rollback

> This scenario is a SUCCESS, not a failure. The system worked as designed: it detected a bad deployment and rolled back automatically before 95% of users were affected. Without Argo Rollouts, the bad version would have gone to 100% of traffic, causing a full outage. The automatic rollback is the whole point.

Interview Tips

For Junior Roles

Focus on Q1-Q15. Know Helm chart structure, basic commands, and values. Understand Kustomize base/overlay pattern. Know what ArgoCD and FluxCD do. Be able to explain Helm vs Kustomize.

For Mid-Level Roles

Q1-Q36. Be ready to write Helm templates, discuss hooks and dependencies, explain ArgoCD sync strategies, and compare Istio vs Linkerd. Know Helm secrets management and CI/CD integration.

For Senior Roles

All 55 questions. GitOps architecture, multi-cluster management, progressive delivery, and service mesh at scale separate senior from mid-level. Be ready to design a GitOps pipeline and discuss repo structure trade-offs.

Quick Reference Cheat Sheet

Topic	Key Takeaway
Helm	Package manager for K8s. Charts, releases, values. --atomic for production.
Helm values	defaults < -f file < --set. Always pin dependency versions.
Helm hooks	Pre/post install/upgrade. Database migrations before deploy.
Kustomize	Patch-based. No templates. Built into kubectl. Base + overlays.
Helm vs Kustomize	Helm for third-party charts. Kustomize for internal apps. Both together is common.
ArgoCD	GitOps with rich UI. Auto-sync, self-heal, multi-cluster. ApplicationSets for scale.
FluxCD	GitOps toolkit. CLI-first. HelmRelease CRD. Image automation built-in.
ArgoCD vs Flux	ArgoCD for UI and multi-cluster. Flux for K8s-native simplicity.
Istio	Full-featured mesh. Envoy sidecar. Canary, circuit breaker, mTLS, AuthPolicy.
Linkerd	Lightweight mesh. Rust proxy. Simple. Lower overhead. "It just works."
OpenShift	Enterprise K8s. SCCs more restrictive. Routes, S2I, OLM, ImageStreams.
GitOps secrets	ESO for Vault/cloud. Sealed Secrets for simple. SOPS for Helm values.
GitOps repo	Separate app and config repos. Directory-based environment promotion.
Argo Rollouts	Progressive delivery. Canary with Prometheus analysis. Auto-rollback.
PENDING_UPGRADE	helm rollback to last good revision. --atomic prevents this.
Self-heal revert	Disable auto-sync → apply hotfix → commit to Git → re-enable. Document the process.
SCC blocking	Use non-root images. anyuid SCC only when necessary with justification.